

Chapitre 2

Systèmes linéaires : méthodes directes

Introduction

Que l'on discrétise une poutre par différences finies, que l'on assemble une matrice de rigidité par éléments finis, ou que l'on modélise un réseau de composants mécaniques linéaires, on aboutit toujours à la même brique de base : un système linéaire $Ax = b$. Ce chapitre présente les **méthodes directes** de résolution – celles qui, en l'absence d'erreur d'arrondi, donneraient la solution exacte en un nombre fini d'opérations. Nous verrons d'abord la résolution des systèmes triangulaires (descente et remontée), puis l'élimination de Gauss et son extension, la méthode de Gauss-Jordan.

🎯 Objectifs pédagogiques

- Résoudre un système triangulaire par descente ou remontée.
- Mettre en œuvre l'élimination de Gauss avec pivotage partiel.
- Mettre en œuvre la méthode de Gauss-Jordan, pour résoudre $Ax = b$ ou pour inverser A .
- Estimer le coût de calcul de chaque méthode et choisir la plus adaptée.
- Implémenter ces méthodes en MATLAB et vérifier leur cohérence avec les solveurs natifs.

Prérequis. Chapitre 1 (normes, conditionnement, stabilité), algèbre linéaire de base (déterminant, matrices triangulaires, matrices symétriques définies positives).

Motivation : pourquoi ce problème existe-t-il ?

La résolution de $Ax = b$ est, de très loin, l'opération numérique la plus fréquemment exécutée en calcul scientifique : elle intervient à chaque pas de temps d'une simulation dynamique, à chaque itération d'un solveur non linéaire (chapitre 4), et au cœur de toute discrétisation par différences finies (Partie II). Une inversion naïve de A (calcul explicite de A^{-1} puis produit $A^{-1}b$) est à la fois plus coûteuse et moins stable numériquement qu'une résolution directe du système – on ne calcule presque jamais A^{-1} en pratique.

Applications en Génie Mécanique

- **Structures** : équilibre statique d'un treillis ou d'une structure discrétisée par éléments finis, $Ku = F$.
- **Thermique** : répartition stationnaire de température après discrétisation par différences finies (chapitres 8-9).

- **Réseaux de composants linéaires** : amortisseurs, ressorts, ou circuits équivalents – exemple fil rouge de ce chapitre.
- **Identification paramétrique** : régression linéaire par moindres carrés, où les équations normales $A^T A x = A^T b$ conduisent à un système linéaire symétrique.

Exemple fil rouge : réseau de trois amortisseurs

Exemple introductif – banc d'essai d'amortisseurs

On considère trois masses alignées, reliées entre elles et à deux bâtis fixes par quatre amortisseurs linéaires (loi $F = c \Delta v$, comme une loi d'Ohm pour des vitesses). En régime permanent, l'équilibre des forces à chaque nœud donne un système linéaire $Cv = F$ où $v = (v_1, v_2, v_3)^T$ sont les vitesses nodales inconnues.

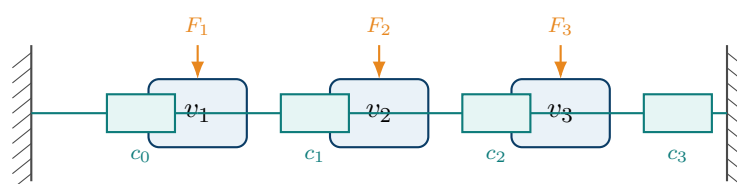


Figure 2.1 – Réseau de trois amortisseurs linéaires c_0, c_1, c_2, c_3 entre deux bâtis fixes, soumis aux forces F_1, F_2, F_3 .

L'équilibre au nœud i s'écrit « somme des forces de rappel des amortisseurs connectés = force extérieure », ce qui donne, par analogie directe avec un réseau de résistances (loi des nœuds) :

$$C = \begin{pmatrix} c_0 + c_1 & -c_1 & 0 \\ -c_1 & c_1 + c_2 & -c_2 \\ 0 & -c_2 & c_2 + c_3 \end{pmatrix}, \quad Cv = F.$$

Remarque

Cette structure – tridiagonale, symétrique – est exactement celle que l'on retrouvera au chapitre 6 pour la discrétisation par différences finies d'une équation différentielle 1D. Ce n'est pas une coïncidence : les deux problèmes traduisent le même principe physique (bilan local reliant chaque nœud à ses voisins immédiats).

Pour la suite du chapitre, on prendra les valeurs numériques $c_0 = 2$, $c_1 = 3$, $c_2 = 1$, $c_3 = 4$ (en N s/m) et $F = (1, 4, 7)^T$ (en N), ce qui donne

$$C = \begin{pmatrix} 5 & -3 & 0 \\ -3 & 4 & -1 \\ 0 & -1 & 5 \end{pmatrix}, \quad F = \begin{pmatrix} 1 \\ 4 \\ 7 \end{pmatrix}.$$

2.1 Position du problème

Étant donné $A \in \mathbb{R}^{n \times n}$ inversible et $b \in \mathbb{R}^n$, on cherche l'unique $x \in \mathbb{R}^n$ tel que $Ax = b$. Une **méthode directe** calcule x en un nombre fini et connu à l'avance d'opérations arithmétiques (par opposition aux méthodes itératives du chapitre 3, qui construisent une suite convergeant vers x).

2.2 Résolution des systèmes triangulaires

Avant d'attaquer le cas général, examinons le cas particulier où A est déjà **triangulaire** – c'est précisément la forme à laquelle l'élimination de Gauss (section suivante) ramène le problème général. Résoudre un système triangulaire coûte $O(n^2)$ opérations, bien moins que le $O(n^3)$ nécessaire pour le mettre sous cette forme.

2.2.1 Système triangulaire supérieur : remontée

Définition – substitution arrière (remontée)

Pour un système $Ux = y$ avec U triangulaire supérieure ($u_{ij} = 0$ si $i > j$) et $u_{ii} \neq 0$ pour tout i , on résout de bas en haut :

$$x_n = \frac{y_n}{u_{nn}}, \quad x_i = \frac{1}{u_{ii}} \left(y_i - \sum_{j=i+1}^n u_{ij} x_j \right), \quad i = n-1, \dots, 1.$$

Exemple – remontée sur un exemple 3×3

Soit $U = \begin{pmatrix} 2 & 1 & -1 \\ 0 & 3 & 2 \\ 0 & 0 & 4 \end{pmatrix}$ et $y = (3, 7, 8)^T$. La remontée donne

$$x_3 = \frac{8}{4} = 2, \quad x_2 = \frac{7 - 2 \cdot 2}{3} = 1, \quad x_1 = \frac{3 - 1 \cdot 1 - (-1) \cdot 2}{2} = 2,$$

soit $x = (2, 1, 2)^T$ – on vérifie aisément que $Ux = y$.

2.2.2 Système triangulaire inférieur : descente

Définition – substitution avant (descente)

Pour un système $Lx = b$ avec L triangulaire inférieure ($\ell_{ij} = 0$ si $i < j$) et $\ell_{ii} \neq 0$ pour tout i , on résout de haut en bas :

$$y_1 = \frac{b_1}{\ell_{11}}, \quad y_i = \frac{1}{\ell_{ii}} \left(b_i - \sum_{j=1}^{i-1} \ell_{ij} y_j \right), \quad i = 2, \dots, n.$$

Exemple – descente sur un exemple 3×3

Soit $L = \begin{pmatrix} 1 & 0 & 0 \\ 3 & 1 & 0 \\ -2 & 4 & 1 \end{pmatrix}$ et $b = (5, 4, 2)^T$. La descente donne

$$y_1 = 5, \quad y_2 = 4 - 3(5) = -11, \quad y_3 = 2 - (-2)(5) - 4(-11) = 2 + 10 + 44 = 56,$$

soit $y = (5, -11, 56)^T$.

Remarque

C'est exactement cette remontée que l'on retrouvera à la fin de l'élimination de Gauss (section suivante) – ces deux substitutions triangulaires, bon marché ($O(n^2)$), sont le « sous-produit utile » de toute triangularisation d'un système linéaire.

2.3 Élimination de Gauss

2.3.1 Principe

L'idée est de transformer le système $Ax = b$ en un système équivalent *triangulaire supérieur* $Ux = y$, facile à résoudre par remontée, en combinant linéairement les lignes du système (opération qui ne change pas l'ensemble des solutions).

Définition – opérations élémentaires sur les lignes

À chaque étape $k = 1, \dots, n - 1$, on annule les coefficients situés sous la diagonale dans la colonne k en effectuant, pour chaque ligne $i > k$:

$$L_i \leftarrow L_i - m_{ik} L_k, \quad m_{ik} = \frac{a_{ik}}{a_{kk}} \quad (\text{multiplicateur de Gauss}),$$

où a_{kk} est le **pivot** de l'étape k .

Attention – pivot nul ou trop petit

Si $a_{kk} = 0$, la méthode échoue (division par zéro). Si a_{kk} est petit devant les autres coefficients, le multiplicateur m_{ik} devient grand et amplifie les erreurs d'arrondi (chapitre 1) : c'est un problème de **stabilité de l'algorithme**, indépendant du conditionnement de A . La parade est le **pivotage partiel** : à chaque étape, on échange la ligne k avec la ligne $i^* \geq k$ qui maximise $|a_{i^*k}|$, ce qui garantit $|m_{ik}| \leq 1$ pour tout i, k et empêche donc les multiplicateurs d'amplifier les erreurs d'arrondi. L'analyse fine de cette stabilité (facteur de croissance des coefficients) est développée dans [4].

Exemple – élimination sur le réseau d'amortisseurs

Appliquons l'élimination à $Cv = F$ (aucun pivotage n'est nécessaire ici, les pivots successifs 5 et 2.2 restent les plus grands en valeur absolue dans leur colonne) :

- Étape 1, pivot $a_{11} = 5$, multiplicateur $m_{21} = -3/5 = -0.6$: $L_2 \leftarrow L_2 + 0.6 L_1$ donne $[0, 2.2, -1 \mid 4.6]$. La ligne 3 est inchangée ($a_{31} = 0$).
- Étape 2, pivot $a_{22} = 2.2$, multiplicateur $m_{32} = -1/2.2 \approx -0.4545$: $L_3 \leftarrow L_3 + 0.4545 L_2$ donne $[0, 0, 4.5455 \mid 9.0909]$.

$$\begin{bmatrix} \boxed{5} & -3 & 0 & 1 \\ -3 & 4 & -1 & 4 \\ 0 & -1 & 5 & 7 \end{bmatrix} \xrightarrow{L_2 \leftarrow L_2 + \frac{3}{5}L_1} \begin{bmatrix} 5 & -3 & 0 & 1 \\ 0 & \boxed{2.2} & -1 & 4.6 \\ 0 & -1 & 5 & 7 \end{bmatrix}$$

Étape 1 : pivot $a_{11} = 5$
 (mis en évidence)

Étape 2 : pivot $a_{22} = 2.2$
 (élimination sous la diagonale)

Figure 2.2 – Les deux étapes de l'élimination de Gauss sur le système du réseau d'amortisseurs.

La remontée sur le système triangulaire $Uv = y$ obtenu donne alors

$$v_3 = \frac{9.0909}{4.5455} = 2, \quad v_2 = \frac{4.6 - (-1)(2)}{2.2} = 3, \quad v_1 = \frac{1 - (-3)(3) - 0(2)}{5} = 2,$$

soit $v = (2, 3, 2)^T$ m/s – solution que nous retrouverons par toutes les méthodes de ce chapitre.

☰ Formulaire – coût de l'élimination de Gauss

À l'étape k , mettre à jour les $n - k$ lignes restantes coûte environ $2(n - k)^2$ opérations (un produit et une soustraction par coefficient modifié). Le coût total de la phase d'élimination est donc

$$\sum_{k=1}^{n-1} 2(n - k)^2 = 2 \sum_{j=1}^{n-1} j^2 \approx \frac{2n^3}{3},$$

auquel s'ajoute un coût $O(n^2)$ pour la remontée triangulaire, négligeable devant n^3 pour n grand.

2.4 Méthode de Gauss-Jordan

2.4.1 Principe

☰ Définition – élimination totale

La méthode de Gauss-Jordan pousse l'élimination plus loin que Gauss : à chaque étape k , on annule les coefficients de la colonne k **au-dessus et en-dessous** du pivot (et non plus seulement en-dessous), puis on normalise la ligne k en la divisant par son pivot. Appliquée au système augmenté $[A | b]$, cette procédure aboutit directement à

$$[A | b] \longrightarrow [I | x],$$

c'est-à-dire que la solution x apparaît directement dans la dernière colonne, **sans étape de remontée séparée** (contrairement à l'élimination de Gauss simple de la section précédente).

🏠 Exemple – Gauss-Jordan sur le système 2×2 déjà rencontré

Reprenons $\begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 5 \\ 11 \end{pmatrix}$ (même système que le premier exercice résolu de ce chapitre), en éliminant maintenant à la fois en-dessous *et* au-dessus de chaque pivot :

$$\left[\begin{array}{cc|c} 2 & 1 & 5 \\ 4 & 3 & 11 \end{array} \right] \xrightarrow{L_1 \leftarrow L_1/2} \left[\begin{array}{cc|c} 1 & 0.5 & 2.5 \\ 4 & 3 & 11 \end{array} \right] \xrightarrow{L_2 \leftarrow L_2 - 4L_1} \left[\begin{array}{cc|c} 1 & 0.5 & 2.5 \\ 0 & 1 & 1 \end{array} \right]$$

$$\xrightarrow{L_1 \leftarrow L_1 - 0.5L_2} \left[\begin{array}{cc|c} 1 & 0 & 2 \\ 0 & 1 & 1 \end{array} \right].$$

On lit directement la solution $x_1 = 2$, $x_2 = 1$ – identique à celle obtenue par Gauss puis remontée, mais sans étape supplémentaire.

2.4.2 Inversion de matrice par Gauss-Jordan

Définition – calcul de A^{-1}

En appliquant exactement le même algorithme au système augmenté $[A | I_n]$ (la matrice identité remplace le second membre b), on obtient

$$[A | I_n] \longrightarrow [I_n | A^{-1}].$$

Exemple – inversion de la matrice 2×2 précédente

$$\left[\begin{array}{cc|cc} 2 & 1 & 1 & 0 \\ 4 & 3 & 0 & 1 \end{array} \right] \xrightarrow{\text{Gauss-Jordan}} \left[\begin{array}{cc|cc} 1 & 0 & 1.5 & -0.5 \\ 0 & 1 & -2 & 1 \end{array} \right], \quad \text{donc} \quad A^{-1} = \begin{pmatrix} 1.5 & -0.5 \\ -2 & 1 \end{pmatrix}.$$

$$\text{Vérification : } AA^{-1} = \begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix} \begin{pmatrix} 1.5 & -0.5 \\ -2 & 1 \end{pmatrix} = \begin{pmatrix} 3-2 & -1+1 \\ 6-6 & -2+3 \end{pmatrix} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix}. \checkmark$$

Remarque

Sur le réseau d'amortisseurs (matrice C de l'exemple fil rouge), la même procédure donne

$$C^{-1} = \begin{pmatrix} 0.38 & 0.30 & 0.06 \\ 0.30 & 0.50 & 0.10 \\ 0.06 & 0.10 & 0.22 \end{pmatrix}.$$

Attention – Gauss-Jordan est plus coûteux que Gauss suivi d'une remontée

Le coût de Gauss-Jordan pour résoudre un seul système est d'environ n^3 opérations (élimination *au-dessus et en-dessous* à chaque étape), contre $\approx 2n^3/3$ pour Gauss suivi d'une remontée – environ **50 % plus cher**. Gauss-Jordan n'est donc **pas** la méthode recommandée pour résoudre $Ax = b$ en pratique. Son intérêt est essentiellement de calculer explicitement A^{-1} – ce que l'on évite en général pour résoudre un système (cf. section « Motivation » en début de chapitre), sauf lorsque l'inverse elle-même présente un intérêt (par exemple pour étudier la sensibilité $\partial x / \partial b = A^{-1}$, chapitre 1).

2.5 Algorithme et organigramme

Algorithme 2: Élimination de Gauss avec pivotage partiel

Donnees: $A \in \mathbb{R}^{n \times n}$, $b \in \mathbb{R}^n$
Resultat: x tel que $Ax = b$

```

1 pour  $k = 1$  à  $n - 1$  faire
2   Trouver  $i^* = \arg \max_{i \geq k} |a_{ik}|$ ;
3   Échanger les lignes  $k$  et  $i^*$  dans  $A$  et  $b$ ;
4   pour  $i = k + 1$  à  $n$  faire
5      $m \leftarrow a_{ik}/a_{kk}$ ;
6      $A(i, :) \leftarrow A(i, :) - m \cdot A(k, :)$ ;
7      $b(i) \leftarrow b(i) - m \cdot b(k)$ ;
8  $x(n) \leftarrow b(n)/a_{nn}$ ;
9 pour  $i = n - 1$  à  $1$  (pas  $-1$ ) faire
10   $x(i) \leftarrow (b(i) - \sum_{j=i+1}^n a_{ij}x(j)) / a_{ii}$ ;
11 retourner  $x$ ;

```

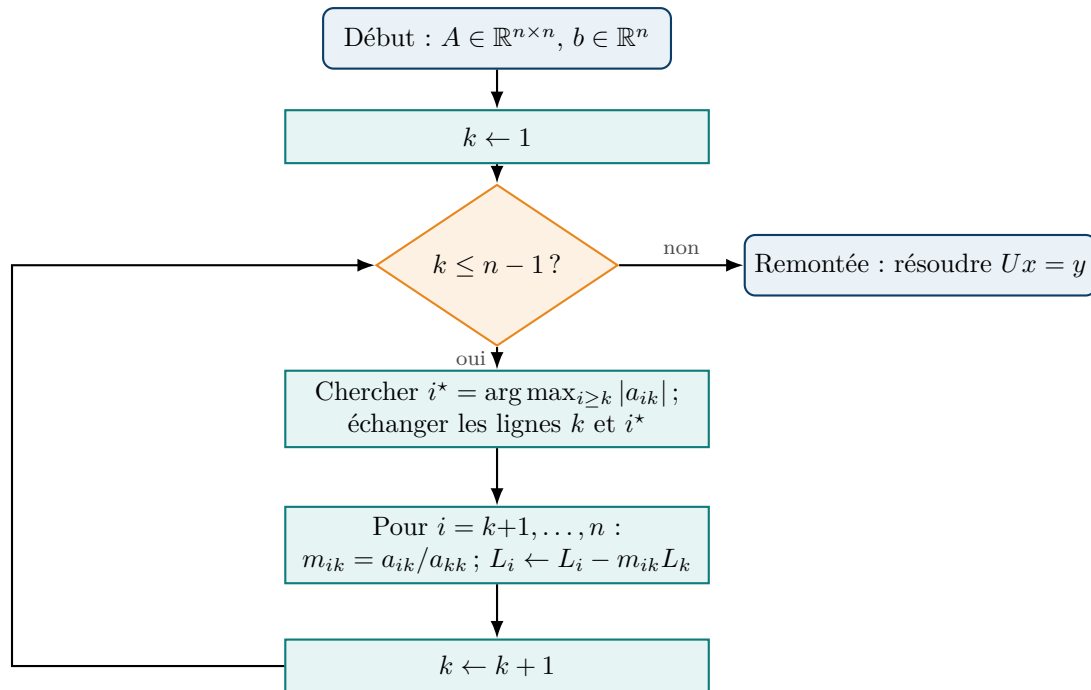


Figure 2.3 – Organigramme de l'élimination de Gauss avec pivotage partiel.

Algorithme 3: Gauss-Jordan (résolution ou inversion)**Donnees:** $A \in \mathbb{R}^{n \times n}$, second membre B ($B = b$ pour résoudre, $B = I_n$ pour inverser)**Resultat:** X tel que $AX = B$ (donc $X = A^{-1}B$)

```

1 Aug  $\leftarrow [A \mid B]$ ;
2 pour  $k = 1$  à  $n$  faire
3   Éventuellement échanger les lignes (pivotage partiel);
4   Aug( $k, :$ )  $\leftarrow$  Aug( $k, :$ )/Aug( $k, k$ ) ;           // normalisation du pivot
5   pour  $i = 1$  à  $n$ ,  $i \neq k$  faire
6     Aug( $i, :$ )  $\leftarrow$  Aug( $i, :$ ) - Aug( $i, k$ ) · Aug( $k, :$ ) ;   // élimination au-dessus et
    en-dessous
7 retourner la partie droite de Aug (c'est  $X$ );

```

2.6 Implémentation MATLAB

MATLAB – Gauss, descente/remontée, Gauss-Jordan sur le réseau d'amortisseurs

```

1 %% chap02_gauss_jordan.m
2 % Reseau de trois amortisseurs : Gauss (pivot partiel),
   descente/remontee, Gauss-Jordan
3 % ATTENTION : en MATLAB, les fonctions locales d'un script doivent
   impérativement
4 % etre placees A LA FIN du fichier (voir bloc en bas).
5 clear; close all; clc;
6
7 %% Donnees du probleme
8 c0=2; c1=3; c2=1; c3=4;           % coefficients d'amortissement
   (N.s/m)
9 C = [c0+c1 -c1 0; -c1 c1+c2 -c2; 0 -c2 c2+c3];
10 F = [1;4;7];
11
12 fprintf('Matrice C :\n'); disp(C);
13 fprintf('Solution de reference (backslash MATLAB) :\n');
14 v_ref = C\F;
15 disp(v_ref');
16
17 %% 1) Elimination de Gauss avec pivotage partiel (implementation
   maison)
18 v_gauss = gauss_pivot(C,F);
19 fprintf('Solution par Gauss (pivot partiel)      : ');
   disp(v_gauss');
20 fprintf('Erreur residuelle ||Cv-F||_2           : %.3e\n',
   norm(C*v_gauss-F));
21
22 %% 1bis) Descente et remontee sur les systemes triangulaires de la
   section 2.2
23 U = [2 1 -1; 0 3 2; 0 0 4]; y = [3;7;8];   % attendu : x = (2,1,2)
24 fprintf('Remontee U x = y, x = '); disp(remontee(U,y));
25
26 L = [1 0 0; 3 1 0; -2 4 1]; b = [5;4;2];    % attendu : y =
   (5, -11, 56)
27 fprintf('Descente L y = b, y = '); disp(descente(L,b));
28

```

```

29 %% 1ter) Gauss-Jordan : resolution directe et inversion de matrice
30 v_gj = gauss_jordan(C, F);
31 Cinv = gauss_jordan(C, eye(3));
32 fprintf('Solution par Gauss-Jordan : '); disp(v_gj');
33 fprintf('Inverse C^-1 par Gauss-Jordan :\n'); disp(Cinv);
34 fprintf('Verification C*C^-1 = I, erreur max : %.3e\n',
    max(max(abs(C*Cinv - eye(3)))));
35
36 %% 2) Etude de complexite : temps de calcul en fonction de n
37 tailles = [50 100 200 400 800];
38 temps = zeros(size(tailles));
39 for k = 1:length(tailles)
40     n = tailles(k);
41     Atest = rand(n) + n*eye(n); % matrice bien conditionnee
42     btest = rand(n,1);
43     tic; Atest\btest; temps(k) = toc;
44 end
45 figure;
46 loglog(tailles, temps, 'o-', 'LineWidth', 1.5); hold on;
47 loglog(tailles, temps(end)*(tailles/tailles(end)).^3, '--');
48 xlabel('taille n'); ylabel('temps (s)');
49 legend('mesure', 'reference n^3', 'Location', 'northwest');
50 title('Complexite empirique de la resolution directe');
51 grid on;
52
53 %% ===== FONCTIONS LOCALES (obligatoirement en fin de
    fichier) =====
54 function x = gauss_pivot(A,b)
55     n = length(b);
56     Aug = [A b];
57     for k = 1:n-1
58         [~,idx] = max(abs(Aug(k:n,k)));
59         idx = idx + k - 1;
60         Aug([k idx], :) = Aug([idx k], :); % echange de lignes
61         for i = k+1:n
62             m = Aug(i,k)/Aug(k,k);
63             Aug(i,:) = Aug(i,:) - m*Aug(k,:);
64         end
65     end
66     x = zeros(n,1);
67     x(n) = Aug(n,end)/Aug(n,n);
68     for i = n-1:-1:1
69         x(i) = (Aug(i,end) - Aug(i,i+1:n)*x(i+1:n)) / Aug(i,i);
70     end
71 end
72
73 function y = descente(L,b)
74     n = length(b); y = zeros(n,1);
75     y(1) = b(1)/L(1,1);
76     for i = 2:n
77         y(i) = (b(i) - L(i,1:i-1)*y(1:i-1)) / L(i,i);
78     end
79 end
80
81 function x = remontee(U,y)
82     n = length(y); x = zeros(n,1);
83     x(n) = y(n)/U(n,n);

```

```

84     for i = n-1:-1:1
85         x(i) = (y(i) - U(i,i+1:n)*x(i+1:n)) / U(i,i);
86     end
87 end
88
89 function X = gauss_jordan(A,B)
90     n = size(A,1);
91     Aug = [A B];
92     for k = 1:n
93         [~,idx] = max(abs(Aug(k:n,k)));
94         idx = idx + k - 1;
95         Aug([k idx],:) = Aug([idx k],:);
96         Aug(k,:) = Aug(k,+)/Aug(k,k);
97         for i = 1:n
98             if i ~= k
99                 Aug(i,:) = Aug(i,:) - Aug(i,k)*Aug(k,:);
100             end
101         end
102     end
103     X = Aug(:, n+1:end);
104 end

```

Interprétation physique des résultats

Les deux méthodes générales (Gauss avec pivotage partiel puis remontée, et Gauss-Jordan) donnent exactement la même solution $v = (2, 3, 2)^T$ m/s, aux erreurs d'arrondi près (résidu $\|Cv - F\|_2$ de l'ordre de 10^{-15}) : c'est normal, ce sont deux formulations du même calcul exact. Les fonctions `descente` et `remontee`, testées séparément sur les deux systèmes triangulaires de la section 2.2, redonnent bien $x = (2, 1, 2)^T$ et $y = (5, -11, 56)^T$: ce sont ces deux briques élémentaires que la triangularisation de Gauss permet d'utiliser sur un système quelconque. L'inverse C^{-1} calculée par Gauss-Jordan vérifie $CC^{-1} = I$ à 10^{-16} près. La courbe de complexité confirme la loi $O(n^3)$ prédite théoriquement – doubler la taille du système multiplie le temps de calcul par environ $2^3 = 8$.

Avantages, limites et pièges

- **Avantage** : les méthodes directes donnent une solution en un nombre d'opérations connu à l'avance, sans risque de non-convergence (contrairement aux méthodes itératives du chapitre 3).
- **Limite** : le coût $O(n^3)$ devient prohibitif pour les très grands systèmes creux (maillages 3D fins) – c'est précisément la motivation des méthodes itératives.
- **Piège** : utiliser Gauss-Jordan (ou pire, calculer A^{-1} puis $A^{-1}b$) pour résoudre un simple $Ax = b$ – c'est à la fois plus coûteux et moins stable numériquement qu'une résolution directe (Gauss suivi d'une remontée). Ne calculer A^{-1} que si l'inverse elle-même est requise.

2.7 Exercices résolus

✂ Exercice résolu – Gauss sans pivotage sur un système 2×2

Énoncé. Résoudre $\begin{pmatrix} 2 & 1 \\ 4 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 5 \\ 11 \end{pmatrix}$ par élimination de Gauss.

Solution. Pivot $a_{11} = 2$, multiplicateur $m_{21} = 4/2 = 2 : L_2 \leftarrow L_2 - 2L_1$ donne $[0, 3-2, | 11-10] = [0, 1|1]$. Remontée : $x_2 = 1$, puis $x_1 = (5 - 1 \cdot 1)/2 = 2$. Solution : $(x_1, x_2) = (2, 1)$.

✂ Exercice résolu – Gauss avec pivotage partiel : quand le pivot est nul

Énoncé. Résoudre par élimination de Gauss avec pivotage partiel

$$A = \begin{pmatrix} 0 & 2 & 1 \\ 1 & 1 & 1 \\ 2 & 1 & 3 \end{pmatrix}, \quad b = \begin{pmatrix} 7 \\ 6 \\ 13 \end{pmatrix}.$$

Solution. Le premier pivot naturel vaut $a_{11} = 0$: sans échange de lignes, l'algorithme calculerait $m_{21} = 1/0$ et s'arrêterait. Le pivotage n'est donc pas ici un raffinement de précision, il est **obligatoire**.

Étape 1. On cherche le plus grand module dans la colonne 1 : $\max(0, 1, 2) = 2$, en ligne 3. On échange $L_1 \leftrightarrow L_3$, puis on élimine avec $m_{21} = 1/2$ et $m_{31} = 0$:

$$\left[\begin{array}{ccc|c} 2 & 1 & 3 & 13 \\ 1 & 1 & 1 & 6 \\ 0 & 2 & 1 & 7 \end{array} \right] \longrightarrow \left[\begin{array}{ccc|c} 2 & 1 & 3 & 13 \\ 0 & 0,5 & -0,5 & -0,5 \\ 0 & 2 & 1 & 7 \end{array} \right].$$

Étape 2. Dans la colonne 2, sous la diagonale, $\max(0,5; 2) = 2$ en ligne 3 : on échange à nouveau, $L_2 \leftrightarrow L_3$. Avec $m_{32} = 0,5/2 = 0,25$:

$$\left[\begin{array}{ccc|c} 2 & 1 & 3 & 13 \\ 0 & 2 & 1 & 7 \\ 0 & 0,5 & -0,5 & -0,5 \end{array} \right] \longrightarrow \left[\begin{array}{ccc|c} 2 & 1 & 3 & 13 \\ 0 & 2 & 1 & 7 \\ 0 & 0 & -0,75 & -2,25 \end{array} \right].$$

Remontée. $x_3 = -2,25/(-0,75) = 3$; $x_2 = (7 - 1 \cdot 3)/2 = 2$; $x_1 = (13 - 1 \cdot 2 - 3 \cdot 3)/2 = 1$. Solution : $x = (1, 2, 3)^T$, que l'on vérifie en reportant dans Ax .

À retenir : on échange *deux fois* ici, et le pivotage partiel ne coûte que la recherche d'un maximum par colonne – $O(n^2)$ comparaisons au total, négligeables devant les $2n^3/3$ opérations de l'élimination. Il n'y a donc jamais de raison de s'en passer.

✂ Exercice résolu – Petit pivot : le pivotage n'est pas un luxe

Énoncé. On résout en double précision (chapitre 1) le système

$$\begin{pmatrix} \delta & 1 \\ 1 & 1 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}, \quad \delta = 10^{-20}.$$

(a) Donner la solution exacte et $\text{cond}_\infty(A)$. (b) Appliquer Gauss *sans* pivotage, en arithmétique flottante. (c) Recommencer *avec* pivotage partiel. Conclure.

Solution. (a) *Le problème.* On vérifie que $x_1 = \frac{1}{1-\delta}$ et $x_2 = \frac{1-2\delta}{1-\delta}$ conviennent : $x_1 \approx$

$x_2 \approx 1$ à 10^{-20} près. Avec $A^{-1} = \frac{1}{\delta-1} \begin{pmatrix} 1 & -1 \\ -1 & \delta \end{pmatrix}$, on obtient $\|A\|_\infty = 2$, $\|A^{-1}\|_\infty = \frac{2}{1-\delta}$, donc $\text{cond}_\infty(A) \approx 4$: le problème est **excellamment conditionné**.

(b) *Sans pivotage*. Le pivot vaut δ , le multiplicateur $m_{21} = 1/\delta = 10^{20}$. L'élimination demande

$$a_{22} = 1 - 10^{20}, \quad b_2 = 2 - 10^{20}.$$

Or au voisinage de 10^{20} (compris entre 2^{66} et 2^{67}), deux flottants consécutifs sont espacés de $2^{66-52} = 2^{14} = 16\,384$. Retrancher 1 ou 2 reste très en deçà d'une demi-case : l'arrondi rend $a_{22} = b_2 = -10^{20}$. La remontée donne alors $x_2 = 1$, puis

$$x_1 = \frac{1 - 1 \cdot x_2}{\delta} = \frac{0}{\delta} = 0 \quad \text{au lieu de } 1 : 100\% \text{ d'erreur.}$$

(c) *Avec pivotage*. $|1| > |\delta|$: on échange $L_1 \leftrightarrow L_2$. Le multiplicateur devient $m_{21} = \delta = 10^{-20}$, et

$$a_{22} = 1 - 10^{-20} \rightarrow 1, \quad b_2 = 1 - 2 \cdot 10^{-20} \rightarrow 1,$$

car cette fois c'est 10^{-20} qui est très en deçà de la demi-case au voisinage de 1 ($2^{-53} \approx 1,1 \times 10^{-16}$). D'où $x_2 = 1$ et $x_1 = (2 - x_2)/1 = 1$: le bon résultat, à la précision machine.

La dégradation est progressive. Sans pivotage, la valeur calculée de x_1 perd des chiffres à mesure que le pivot rétrécit, jusqu'à la perte totale :

δ	10^{-10}	10^{-12}	10^{-14}	10^{-16}	10^{-17} à 10^{-20}
x_1 calculé	1,000 000 08	0,999 98	0,999 2	2,22	0
erreur relative	8×10^{-8}	2×10^{-5}	8×10^{-4}	122 %	100 %

À retenir : même matrice, même machine, $\text{cond}_\infty(A) = 4$. L'échec de (b) ne vient donc pas du problème, il vient de l'algorithme : un multiplicateur de 10^{20} fait disparaître l'information à l'arrondi. C'est exactement la distinction *conditionnement / stabilité* du chapitre 1, et c'est ce que le pivotage partiel empêche en garantissant $|m_{ik}| \leq 1$. (Vérifié en MATLAB : le calcul (b) rend $x = (0, 1)$, et $A \backslash b$ rend $(1, 1)$.)

✂ Exercice résolu – Descente sur un système triangulaire inférieur

Énoncé. Résoudre $\begin{pmatrix} 2 & 0 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} y_1 \\ y_2 \end{pmatrix} = \begin{pmatrix} 6 \\ 1 \end{pmatrix}$ par descente.

Solution. $y_1 = 6/2 = 3$, puis $y_2 = (1 - 3 \cdot 3)/4 = (1 - 9)/4 = -2$. Solution : $(y_1, y_2) = (3, -2)$.

✂ Exercice résolu – Gauss-Jordan sur le réseau d'amortisseurs, en fractions exactes

Énoncé. Résoudre le système fil rouge $Cv = F$ par Gauss-Jordan avec pivotage partiel, en conservant des fractions exactes. Vérifier que l'on retrouve $v = (2, 3, 2)^T$ sans remontée, et relier les pivots obtenus à ceux de l'élimination de Gauss.

Solution. On part de la matrice augmentée

$$\left[\begin{array}{ccc|c} 5 & -3 & 0 & 1 \\ -3 & 4 & -1 & 4 \\ 0 & -1 & 5 & 7 \end{array} \right].$$

Étape 1. Colonne 1 : $\max(|5|, |-3|, |0|) = 5$, déjà en place, pas d'échange. On normalise

$L_1 \leftarrow L_1/5$, puis $L_2 \leftarrow L_2 + 3L_1$ (L_3 est inchangée, car $a_{31} = 0$) :

$$\left[\begin{array}{ccc|c} 1 & -\frac{3}{5} & 0 & \frac{1}{5} \\ 0 & \frac{11}{5} & -1 & \frac{23}{5} \\ 0 & -1 & 5 & 7 \end{array} \right].$$

Étape 2. Colonne 2, sous la diagonale : $\max(\frac{11}{5}, |-1|) = \frac{11}{5}$, pas d'échange. On normalise $L_2 \leftarrow \frac{5}{11}L_2$, puis on élimine *au-dessus et en-dessous* : $L_1 \leftarrow L_1 + \frac{3}{5}L_2$ et $L_3 \leftarrow L_3 + L_2$:

$$\left[\begin{array}{ccc|c} 1 & 0 & -\frac{3}{11} & \frac{16}{11} \\ 0 & 1 & -\frac{5}{11} & \frac{23}{11} \\ 0 & 0 & \frac{50}{11} & \frac{100}{11} \end{array} \right].$$

Étape 3. $L_3 \leftarrow \frac{11}{50}L_3$ donne $[0 \ 0 \ 1 \ | \ 2]$; puis $L_1 \leftarrow L_1 + \frac{3}{11}L_3$ et $L_2 \leftarrow L_2 + \frac{5}{11}L_3$:

$$\left[\begin{array}{ccc|c} 1 & 0 & 0 & 2 \\ 0 & 1 & 0 & 3 \\ 0 & 0 & 1 & 2 \end{array} \right] \implies v = (2, 3, 2)^T \text{ m/s,}$$

lu directement dans la dernière colonne, comme par Gauss suivi d'une remontée.

À retenir : les pivots rencontrés, 5, $\frac{11}{5} = 2,2$ et $\frac{50}{11} \approx 4,545$, sont *exactement* ceux de l'élimination de Gauss de la section précédente – et leur produit $5 \times \frac{11}{5} \times \frac{50}{11} = 50$ redonne $\det C$. Gauss-Jordan fait donc le même travail sous la diagonale, *plus* les éliminations au-dessus (une à l'étape 2, deux à l'étape 3) : c'est l'origine de son surcoût d'environ 50 %.

✂ Exercice résolu – Gauss-Jordan pour inverser une matrice 2×2

Énoncé. Calculer B^{-1} pour $B = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ par Gauss-Jordan.

Solution.

$$\begin{aligned} & \left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 3 & 4 & 0 & 1 \end{array} \right] \xrightarrow{L_2 \leftarrow L_2 - 3L_1} \left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & -2 & -3 & 1 \end{array} \right] \\ & \xrightarrow{L_2 \leftarrow L_2 / (-2)} \left[\begin{array}{cc|cc} 1 & 2 & 1 & 0 \\ 0 & 1 & 1.5 & -0.5 \end{array} \right] \xrightarrow{L_1 \leftarrow L_1 - 2L_2} \left[\begin{array}{cc|cc} 1 & 0 & -2 & 1 \\ 0 & 1 & 1.5 & -0.5 \end{array} \right]. \end{aligned}$$

Donc $B^{-1} = \begin{pmatrix} -2 & 1 \\ 1.5 & -0.5 \end{pmatrix}$. Vérification : $\det B = 4 - 6 = -2$, et la formule directe $B^{-1} = \frac{1}{\det B} \begin{pmatrix} 4 & -2 \\ -3 & 1 \end{pmatrix} = \begin{pmatrix} -2 & 1 \\ 1.5 & -0.5 \end{pmatrix}$. ✓

✂ Exercice résolu – Choisir la méthode selon le nombre de seconds membres

Énoncé. Une structure discrétisée conduit à une matrice de rigidité K d'ordre $n = 1000$. On doit calculer le déplacement pour **50 cas de charge** $F_1, \dots, F_{50}$, c'est-à-dire résoudre 50 systèmes qui partagent la *même* matrice. Comparer deux stratégies et conclure.

Solution. Le point clé est que l'élimination ne dépend que de K : elle est identique pour les 50 cas. Seule la remontée change d'un cas à l'autre.

Stratégie 1 – relancer Gauss 50 fois. Chaque résolution coûte $\approx 2n^3/3$:

$$50 \times \frac{2 \times (10^3)^3}{3} \approx 50 \times 6,7 \times 10^8 \approx 3,3 \times 10^{10} \text{ opérations.}$$

Stratégie 2 – éliminer une seule fois. On applique l'élimination à la matrice augmentée $[K | F_1 F_2 \dots F_{50}]$, en faisant subir à chaque second membre les mêmes combinaisons de lignes. On paie l'élimination *une* fois, puis 50 remontées à $\approx n^2$ opérations :

$$\frac{2n^3}{3} + 50n^2 \approx 6,7 \times 10^8 + 5,0 \times 10^7 \approx 7,2 \times 10^8 \text{ opérations.}$$

Stratégie	opérations	temps à 1 Gflop/s
50 éliminations séparées	$3,3 \times 10^{10}$	33 s
1 élimination + 50 remontées	$7,2 \times 10^8$	0,7 s

Gain : un facteur 46, pour exactement le même résultat. La raison est arithmétique : la partie chère croît en n^3 et n'est faite qu'une fois, tandis que la partie répétée ne coûte que n^2 – mille fois moins ici.

Et l'inverse ? Calculer K^{-1} puis faire 50 produits $K^{-1}F_j$ semble séduisant, mais l'inversion par Gauss-Jordan coûte à elle seule plus cher que la stratégie 2 tout entière – et l'on a vu au chapitre 1 qu'elle dégrade la précision. La règle du chapitre tient donc toujours : **on n'inverse pas pour résoudre.**

✂ Exercice résolu – Vérifier un code MATLAB : la remontée qui ne remonte pas

Énoncé. Un étudiant programme la remontée ainsi :

```

1 function x = remontee(U,y)
2     n = length(y); x = zeros(n,1);
3     x(n) = y(n)/U(n,n);
4     for i = n-1:1
5         x(i) = (y(i) - U(i,i+1:n)*x(i+1:n)) / U(i,i);
6     end
7 end

```

Il le teste sur le système triangulaire de la section 2.2 (U et $y = (3, 7, 8)^T$, solution attendue $x = (2, 1, 2)^T$). MATLAB affiche $x = [0; 0; 2]$, sans erreur ni avertissement. (a) Sans connaître la solution, quel test aurait détecté le problème ? (b) Trouver l'erreur. (c) Pourquoi la dernière composante est-elle juste ?

Solution. (a) *Deux tests automatiques*, ceux de l'objectif « vérifier la cohérence avec les solveurs natifs ». Le **résidu** d'abord : $Ux = (-2, 4, 8)^T$, donc $Ux - y = (-5, -3, 0)^T$ et $\text{norm}(Ux - y) = \sqrt{34} \approx 5,83$, là où un code correct rend 0 ou quelques 10^{-15} . La **comparaison au solveur natif** ensuite : $U \backslash y$ rend $(2, 1, 2)^T$, soit un écart de 2 sur deux composantes.

(b) *L'erreur.* En MATLAB, $a:b$ progresse par pas de +1. Pour $n = 3$, $n-1:1$ vaut donc $2:1$, un vecteur *vide* (taille 1×0) : la boucle ne s'exécute jamais, et MATLAB ne signale rien. Il faut imposer le pas descendant : **for i = n-1:-1:1**.

(c) *Juste par construction, pas par calcul.* $x_n = y_n/u_{nn}$ est calculé *avant* la boucle : la dernière composante est donc correcte, et les autres restent à zéro, valeur de l'initialisation **zeros(n,1)**.

À retenir : ce code est « juste par accident » sur un petit test. Pour $n = 2$, $n-1:1$ vaut $1:1$, la boucle tourne exactement une fois et le résultat est *exact* : un test sur un système 2×2 n'aurait rien révélé.

On valide un solveur par son **résidu** et sur plusieurs tailles, jamais en regardant la solution.
(Comportement vérifié en MATLAB : zéro itération, $\|Ux - y\|_2 = 5,8310$; avec $\mathbf{n} = 1:-1:1$, résidu nul.)

Résumé du chapitre

Résumé du chapitre

- Un système triangulaire se résout en $O(n^2)$ par substitution : **descente** ($Ly = b$, de haut en bas) ou **remontée** ($Ux = y$, de bas en haut).
- L'élimination de Gauss transforme $Ax = b$ en un système triangulaire équivalent, résolu par remontée – coût $O(2n^3/3)$.
- Le pivotage partiel (échanger les lignes pour maximiser le pivot) assure $|m_{ik}| \leq 1$ et rend l'élimination de Gauss stable en pratique, indépendamment du conditionnement de A – sans toutefois que la stabilité soit garantie au sens strict.
- Gauss-Jordan pousse l'élimination au-dessus *et* en-dessous de chaque pivot : $[A|b] \rightarrow [I|x]$ sans remontée séparée, mais pour un coût $\approx n^3$ (environ 50 % plus cher que Gauss). Son usage principal est le calcul explicite de A^{-1} via $[A|I] \rightarrow [I|A^{-1}]$.

Carte mentale

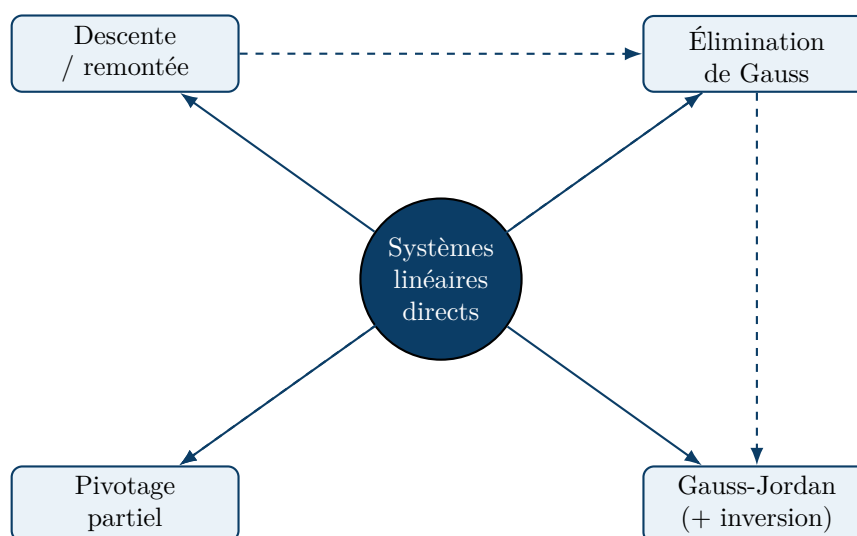


Figure 2.4 – Carte mentale du chapitre 2.

Formulaire

$$\begin{array}{ll}
 \text{Remontée : } x_i = \frac{1}{u_{ii}} \left(y_i - \sum_{j>i} u_{ij} x_j \right) & \text{Descente : } y_i = \frac{1}{\ell_{ii}} \left(b_i - \sum_{j<i} \ell_{ij} y_j \right) \\
 m_{ik} = a_{ik}/a_{kk} \text{ (multiplicateur)} & \text{Coût Gauss} \approx \frac{2n^3}{3} \\
 \text{Gauss-Jordan : } [A|b] \rightarrow [I|x], \text{ coût} \approx n^3 & [A|I_n] \rightarrow [I_n|A^{-1}] \text{ (inversion)}
 \end{array}$$

Quiz

1. Le pivotage partiel sert principalement à : (a) réduire le nombre d'opérations, (b) garantir la stabilité numérique, (c) rendre A symétrique.

2. Le coût de l'élimination de Gauss croît, pour une matrice $n \times n$, comme : (a) $O(n)$, (b) $O(n^2)$, (c) $O(n^3)$.
3. La remontée (résolution de $Ux = y$) se fait : (a) de haut en bas, (b) de bas en haut, (c) dans un ordre arbitraire.
4. Comparé à Gauss suivi d'une remontée, Gauss-Jordan pour résoudre un seul système est : (a) moins cher, (b) environ 50 % plus cher, (c) de même coût.

Réponses : 1b - 2c - 3b - 4b.

Exercices d'entraînement

1. [Facile] Résoudre par Gauss (sans pivotage) le système $\begin{pmatrix} 3 & 1 \\ 6 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 9 \\ 24 \end{pmatrix}$.
2. [Facile] Évaluer le nombre d'opérations approximatif $(2n^3/3)$ de l'élimination de Gauss pour $n = 100$ et $n = 1000$. Combien de fois le temps de calcul est-il multiplié ?
3. [Difficile] On ajoute un quatrième nœud au réseau d'amortisseurs (masse supplémentaire reliée à la masse 3 par un amortisseur c_4 et au bâti par c_5). Écrire la nouvelle matrice 4×4 et donner sa structure (bande, symétrie).
4. [Très difficile] Montrer, à l'aide de la forme quadratique $v^T C v = c_0 v_1^2 + c_1 (v_1 - v_2)^2 + c_2 (v_2 - v_3)^2 + c_3 v_3^2$, que la matrice C du réseau d'amortisseurs est toujours définie positive dès lors que $c_0, c_1, c_2, c_3 > 0$, et interpréter physiquement cette forme quadratique en termes de puissance dissipée.
5. [Facile] Résoudre par remontée le système triangulaire supérieur $\begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} = \begin{pmatrix} 5 \\ 9 \end{pmatrix}$.
6. [Moyen] Calculer B^{-1} par Gauss-Jordan, pour $B = \begin{pmatrix} 4 & 2 & 0 \\ 2 & 3 & 1 \\ 0 & 1 & 2 \end{pmatrix}$.

Astuce

Les corrigés détaillés de ces exercices se trouvent dans `corriges/corrige_chap02.tex`.